ProctorAI AI Based Online Proctoring System

***TeamID: [Write Your Assigned Team ID Here]**

Student Name 1 (Roll No: XXXXXXXX)
Student Name 2 (Roll No: XXXXXXXX)
Student Name 3 (Roll No: XXXXXXXX)
Student Name 4 (Roll No: XXXXXXXX)
Student Name 5 (Roll No: XXXXXXXX)
Student Name 6 (Roll No: XXXXXXXX)
Department of Computer Science Engineering and Technology
Bennett University, India

GitHub Link: https://github.com/MehulMTyagi/PROCTORAI

Abstract - ProctorAI is an AI based online proctoring system designed for college level remote assessments. The problem addressed by the project is that online tests are difficult to supervise because a candidate may use another device, leave the camera frame, receive help, switch applications, or stop monitoring permissions. The proposed system combines a candidate portal, examiner/proctor portal, FastAPI backend, local AI sidecar, browser integrity monitoring, and protected evidence handling. The monitoring design includes face absence detection, multiple person detection, YOLO based phone detection, liveness verification, screen recording checks, copy and paste tracking, fullscreen and focus-loss events, and a planned identity identification step using registered and live candidate face images. Instead of automatically declaring misconduct, the system sends evidence backed integrity events to the proctor for review. The current Phase 2 work focuses on architecture, module integration, dataset planning, and evaluation design; final quantitative results will be updated after complete testing before project submission.

Index Terms - Online proctoring, computer vision, YOLO, face detection, liveness verification, identity identification, integrity events, evidence review.

I. INTRODUCTION

Online examinations are useful because they allow students to appear for tests without being physically present in a classroom. At the same time, they create a serious supervision problem. In a normal examination hall, an invigilator can observe the candidate, check the surroundings, and notice suspicious actions. In an online test, the candidate may use a phone, receive help from another person, leave the camera frame, switch tabs, copy answers, or stop screen recording.

The problem addressed in this project is the lack of reliable and reviewable supervision in online assessments. Many simple online test systems only provide questions and answer submission. They do not provide strong setup checks, candidate identity verification, browser activity tracking, or evidence based proctor review. This makes them weak for serious college examinations.

ProctorAI is proposed as an AI based online proctoring system for college assessments. It combines a candidate portal, examiner/proctor portal, backend API, local AI sidecar, screen recording checks, camera and microphone monitoring, liveness verification, and event evidence review. The system is designed to detect suspicious behavior and send integrity events to a proctor instead of automatically punishing the candidate.

The main contribution of this project is a privacy-aware multi-signal proctoring workflow. The system uses AI and browser signals together: face absence, multiple persons, phone detection, copy and paste, focus lost, fullscreen exit, screen recording stopped, background audio, liveness failure, and identity identification. The proctor receives proof and can review the event before making a decision.

II. LITERATURE SURVEY

AI based proctoring research shows that online examination monitoring usually combines several signals rather than relying on one detector. A systematic review of AI based proctoring systems reported common use of face monitoring, gaze or behavior analysis, audio analysis, and log based activity tracking [1]. This supports the direction of ProctorAI because cheating or suspicious behavior can appear through the camera, browser, microphone, or device state.

Face recognition and face detection are major components of proctoring systems. Surveys on face recognition based exam monitoring discuss methods such as Viola-Jones, CNN based face detectors, and MTCNN [2], [9]. For ProctorAI, face detection is used first as a presence and continuity signal: the system checks whether the candidate is visible, whether multiple people appear, and whether identity verification should be repeated.

Object detection is required for detecting mobile phones and additional persons. YOLO style detectors are widely used for real time detection because they process an image in a single forward pass [5], [6]. COCO is useful as a base object detection dataset because it provides common object categories in natural scenes [7]. However, general object datasets do not fully represent exam room conditions, so a small custom validation set is still needed.

Identity identification and liveness verification are also important for a production grade proctoring system. FaceNet and ArcFace show how face embeddings can be used to compare whether two face images belong to the same person [11], [12]. Face anti-spoofing surveys show that liveness checks are needed because a photo, video replay, or mask can attack a face recognition system [13]. ProctorAI therefore treats identity verification and liveness as separate steps.

Recent discussions on remote proctoring also warn about bias, privacy, and unfair automatic decisions [15]. This is why ProctorAI uses human review: the AI system raises an integrity event with proof, but the final decision remains with the proctor.

III. PROPOSED METHODOLOGY

The proposed methodology follows a multi-module architecture. The candidate first selects an assigned assessment and completes setup checks. These checks include consent, backend connectivity, camera permission, microphone permission, screen recording, connected display audit, liveness verification, and identity

identification. The identity module compares a registered candidate image with a live setup image and sends uncertain cases to the proctor for review.

During the test, the candidate portal monitors browser and media signals. Browser integrity events include copy, paste, fullscreen exit, focus lost, suspicious shortcuts, and stopped screen recording. Camera based events include face absent, multiple persons, phone detected, and liveness failure. Audio events include microphone interruption and background voice or noise signals. Evidence is attached as short clips or proof metadata instead of continuously streaming all media.

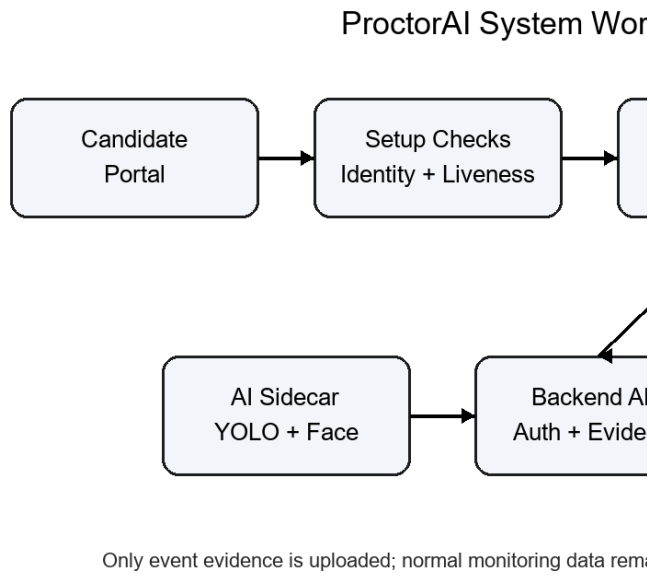


Fig. 1. Proposed ProctorAI system workflow.

The backend stores users, assessments, attempts, answers, integrity events, evidence metadata, and proctor review decisions. A WebSocket or equivalent real time channel sends new integrity events to the proctor portal. The proctor portal displays each event with event type, severity, confidence, timestamp, evidence status, and review action. The final system will support confirmed suspicious, false positive, and inconclusive decisions.

The AI sidecar is responsible for local computer vision inference. YOLO is used for phone and object detection, while face analysis is used for face count, face absence, liveness support, and identity continuity. This architecture keeps most normal monitoring data local and uploads only event specific evidence, which is more privacy aware than continuous cloud recording.

Module	Purpose	Phase 2 Status
Candidate portal	Assignment selection, setup checks, test attempt	Implemented and being refined
Proctor portal	Live event queue and evidence review	Implemented with review workflow planned for final hardening
AI sidecar	Face, liveness, and phone/object analysis	Implemented for core signals
Identity identification	Compare registered and live candidate identity	Included in final feature scope
Evidence handling	Store proof clips and hashes for review	Implemented for event evidence

Table I. Major ProctorAI modules and Phase 2 status.

IV. EXPERIMENTAL RESULTS

A. Dataset Description

The project uses a combination of public datasets, pretrained model knowledge, and custom validation data. COCO is selected for object detection support because it contains common objects such as persons and mobile phones in natural contexts [7]. WIDER FACE is relevant for face detection benchmarking because it contains many face images with variation in scale, pose, and occlusion [8]. A custom validation set will be prepared for exam-like cases such as normal candidate sitting, face absent, multiple persons, phone visible, poor lighting, background movement, and screen recording interruption.

B. Evaluation Metrics

For event detection, the planned evaluation metrics are precision, recall, F1-score, false positive rate, event delivery latency, and evidence availability rate. Precision measures how many generated events are actually relevant. Recall measures how many expected events are detected. F1-score balances precision and recall. Latency measures how quickly the proctor receives an event after it occurs. Evidence availability rate measures whether each event includes usable proof for review.

C. Current Validation and Comparison

Signal	Detector or Source	Expected Output	Evidence Type	Status
Face absent	Camera and face count	FACE_ABSENT event	Camera clip/proof	Implemented
Multiple persons	Camera and face count	MULTIPLE_PERSONS event	Camera clip/proof	Implemented
Phone visible	YOLO object detection	PHONE_DETECTED event	Camera clip/proof	Implemented
Copy or paste	Browser event listener	Clipboard event	Screen clip	Implemented
Identity mismatch	Face comparison	Identity review event	Planned identity proof	Final scope

Table II. Planned and current integrity signal validation.

At this stage, the report records Phase 2 design and module validation rather than final model accuracy. Final numerical performance values will be added after full testing on the custom validation set. This is academically safer than reporting artificial accuracy values before the complete experiment is finished.

D. Hyperparameter Tuning

The main parameters to tune are object detection confidence threshold, consecutive-frame count for event confirmation, liveness motion threshold, audio activity threshold, and identity match threshold. The tuning strategy will compare lenient, moderate, and strict settings on the validation set. The selected configuration should reduce false positives while still detecting important integrity risks.

V. CONCLUSION

This Phase 2 report presents the design and development direction of ProctorAI, an AI based online proctoring system for college assessments. The system addresses the problem of weak supervision in online tests by combining candidate setup checks, identity verification, liveness, object detection, browser integrity monitoring, evidence upload, and proctor review.

The current limitation is that final quantitative accuracy and latency results still need to be measured after completing the full validation dataset and final project testing. Future work before final submission includes completing identity identification, improving hosted deployment reliability, tuning thresholds, and generating final proctoring reports.

REFERENCES

[1] A. Nigam, R. Pasricha, T. Singh, and P. Churi, "A systematic review on AI-based proctoring systems: Past, present and future," Education and Information Technologies, 2021.

[2] R. A. Pramudita et al., "Survey of online exam proctoring model to detect cheating behavior based on face recognition," IEEE Conference Publication, 2022.

[3] A. H. M. et al., "Towards effective and efficient online exam systems using deep learning-based cheating detection approach," Computers and Education: Artificial Intelligence, vol. 3, 2022.

[4] D. Somavarapu et al., "A systematic-narrative review of online proctoring systems and a case for open standards," 2025.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," Proc. CVPR, 2016.

[6] J. Terven and D. Cordova-Esparza, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," arXiv:2304.00501, 2023.

[7] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," Proc. ECCV, 2014.

[8] S. Yang, P. Luo, C. C. Loy, and X. Tang, "WIDER FACE: A face detection benchmark," Proc. CVPR, 2016.

[9] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," IEEE Signal Processing Letters, 2016.

[10] V. Bazarevsky et al., "BlazeFace: Sub-millisecond neural face detection on mobile GPUs," arXiv:1907.05047, 2019.

[11] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," Proc. CVPR, 2015.

[12] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," Proc. CVPR, 2019.

[13] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, "Deep learning for face anti-spoofing: A survey," arXiv:2106.14948, 2021.

[14] T. Baltrusaitis, A. Zadeh, Y. C. Lim, and L.-P. Morency, "OpenFace 2.0: Facial behavior analysis toolkit," Proc. FG, 2018.

[15] M. Burgess et al., "Watching the watchers: Bias and vulnerability in remote proctoring software," USENIX Security Symposium, 2022.